Server Room Monitoring Dashboard - Beginner Explanation

1. Big Picture
Your project is a full stack application with two parts:
- Backend: Django + Django REST Framework
- Frontend: React + Recharts

The idea is simple:
A sensor reading is stored in Django, Django exposes it as JSON through an API, and React reads that JSON to draw charts.

Flow:
Sensor -> Django Model -> Database -> DRF API -> React -> Recharts Chart

Think of it like a server room with a wall display:
- The sensor measures temperature and humidity.
- Django is the office worker that stores the readings.
- The database is the filing cabinet.
- The API is the translator that turns records into JSON.
- React is the dashboard screen.
- Recharts is the pen that draws the graphs.

2. Django Backend

models.py
Your SensorData model represents one reading from the server room.
It stores:
- temperature
- humidity
- created_at

Why this matters:
Each row in the database is one sensor snapshot. If the sensor sends a new value every minute, Django can save one new SensorData row every minute.

Important fields:
- DecimalField is used for temperature and humidity because measurements often need decimals.
- created_at uses auto_now_add=True, which means Django stores the creation time automatically.
- db_index=True helps time-based sorting and searching become faster.

The Meta ordering = ["-created_at"] means the newest reading comes first by default.

serializers.py
The serializer converts Django model objects into JSON.

Why we use it:
Python objects cannot be sent directly to the browser in a useful way. React needs JSON,

so the serializer acts like a translator.

Example idea:
SensorData object in Django -> JSON like:

```
{
  "id": 1,
  "temperature": "24.50",
  "humidity": "52.00",
  "created_at": "2026-03-23T10:30:00Z"
}
```


views.py
Your home view returns a simple JSON message for the root URL.

Your SensorDataViewSet uses DRF ModelViewSet.
That is a ready-made class that automatically gives common API actions like:
- list all sensor readings
- get one reading
- create a reading
- update a reading
- delete a reading

In your code:
- queryset = SensorData.objects.all() means it reads all SensorData rows.
- serializer_class = SensorDataSerializer means it converts those rows using the serializer.

urls.py
At the app level, you use DefaultRouter.
That router automatically creates routes for the viewset.
Registering "sensor-data" creates routes such as:
- /sensor-data/
- /sensor-data/1/

At the project level, you include monitoring.urls under /api/.
That means the final endpoint becomes:
- /api/sensor-data/

So routing works like this:
project urls.py -> /api/ -> monitoring.urls -> router -> SensorDataViewSet

3. Data Flow from Database to Frontend
The backend flow is:
1. A reading exists in the database.
2. Django loads it through the SensorData model.
3. SensorDataViewSet gets the queryset.
4. SensorDataSerializer turns it into JSON.
5. Django REST Framework sends it as an HTTP response.
6. React fetches that response.

7. Recharts draws the chart.

Short version:
Database -> Model -> Serializer -> API -> React -> Chart

4. React Frontend

Dashboard.jsx
This is the main page logic for your dashboard.
It prepares the chart data, calculates the latest values, and sends data to child components like SensorChart.

Right now your dashboard is using mock data, not live backend data.
That is okay for testing because it lets the charts display even when the backend is not connected.

Mock data
You created a const data = [...] array with 20 points.
Each point has:
- time
- temperature
- humidity

This array is used as the starting chart state.

useState
React useState stores values that can change over time.
In your dashboard:
- chartData stores the list of points shown in the charts.
- lastUpdated stores the timestamp of the latest update.

When state changes, React re-renders the component.

useEffect
React useEffect runs side effects such as timers or API calls.
In your current dashboard, useEffect starts a timer with setInterval.
Every 3 seconds it:
- builds a new mock sensor point
- adds it to the chart data
- removes the oldest point so the list stays at 20 items
- updates the lastUpdated time

That creates the feeling of live sensor updates.

How Recharts uses the data
SensorChart receives props like:
- data
- dataKey
- color

- unit

Inside SensorChart:
- <LineChart data={data}> means use the array as the chart source.
- <XAxis dataKey="time" /> uses the time field for the horizontal axis.
- <Line dataKey="temperature" /> draws the temperature values.
- <Line dataKey="humidity" /> draws the humidity values.

This is why one array can power two charts.
Each object contains both temperature and humidity, and each chart reads a different property.

How cards show values
The dashboard also takes the latest point:
- latestPoint.temperature
- latestPoint.humidity

Those are shown in the summary cards so the number cards and the charts are based on the same source of truth.

5. How React Connects to Django
You also have a fetch helper in sensorApi.js.
Its job is to call:
- /api/sensor-data/

It uses fetch() to send an HTTP request.
If the response is successful, it returns response.json().
That JSON can then be stored in React state.

When /api/sensor-data/ is called, the full chain is:
Browser -> Django URL router -> DRF router -> SensorDataViewSet -> SensorData queryset -> Serializer -> JSON response -> React fetch() -> state update -> chart redraw

Important note about your current setup:
Right now the dashboard is in mock mode for testing, so it is not actively using fetchSensorData.
The backend API is ready, but the current chart display is powered by local mock data.

6. Beginner Summary
If you want to imagine the whole project simply, think of this:
- The server room sensor measures the environment.
- Django stores each reading.
- DRF turns the reading into JSON.
- React asks for the JSON.
- Recharts turns the numbers into lines.

Text flow:
Sensor -> Django -> Database -> API -> React -> Chart

7. Mistakes or Improvements
Here are the best improvements for your project:

- Separate mock mode and live API mode clearly.
  Right now the backend exists, but the dashboard uses mock data. A simple USE_MOCK_DATA flag would help.

- Consider ReadOnlyModelViewSet instead of ModelViewSet.
  If the frontend only needs to read data, ReadOnlyModelViewSet is safer because it avoids accidental create, update, or delete actions.

- Limit the queryset.
  Returning every row can become slow later. For charts, usually the latest 20, 50, or 100 records are enough.

- Add validation.
  Prevent impossible values such as humidity above 100 or temperature far outside your expected range.

- Move mock data into a separate file.
  That keeps Dashboard.jsx cleaner and makes switching between test mode and live mode easier.

- Use polling or WebSockets for real-time updates.
  Polling every few seconds is the easiest beginner option. WebSockets are a good next step for true live dashboards.

- Add loading and error states when you reconnect the real backend.
  This makes the dashboard feel more reliable.

8. Architecture Diagram

Current frontend test mode:
Mock data -> useState -> SensorChart -> Recharts lines

Backend mode:
Sensor -> Django Model -> SQLite Database -> Serializer -> /api/sensor-data/ -> React fetch -> useState -> Recharts charts

Full text diagram:

[SENSOR DEVICE]
    |
    v
[Django Model: SensorData]
    |
    v
[SQLite Database]
    |

v
[DRF Serializer]
|
v
[API Endpoint: /api/sensor-data/]
|
v
[React Dashboard]
|
v
[Recharts Temperature and Humidity Charts]

9. Final Takeaway
Your project already has a solid structure:
- Django stores and serves data.
- DRF exposes data cleanly as JSON.
- React manages screen state.
- Recharts visualizes the data.

The biggest thing to understand is that charts are only a visual layer.
The real work happens in the data flow:
Sensor reading -> database record -> API response -> React state -> chart line.

Once that idea is clear, the whole dashboard becomes much easier to understand and extend.